

Time Series Analysis for Data Science

Time Series Data

- > Observations collected over a sequence of time intervals
- > data gathered on the basis of daily data, weekly data, seconds data, monthly data etc.
- > data can be any kind of which will be stock prices, sales data, weather data, social media activity.

1.1 Characteristics of Time Series Data

- > Chronological Order:
 - Time series data is arranged in chronological order, meaning observations are organized according to the time at which they occurred.
- For Example: Consider a company yearly sales

Year	Sales
2022	₹ 10 lakh
2023	₹ 12 lakh
2024	₹ 15 lakh
2025	₹ 18 lakh
- Why is this important?
 - The order of observation matters.
 - Earlier observations occur before later observations.
 - We can study how a variable changes over time.
 - Past values can sometimes help us to understand or predict future values.

↳ Sequential data:

- Time series data is sequential, which means the position or order of each observation is important
- In ordinary data, changing the order of observation may not make much difference. But in Time Series, changing the order can destroy important information.

- for example: correct sequence:

Monday → Tuesday → Wednesday → Thursday → Friday

Suppose the Temperature is:

$25^{\circ}\text{C} \rightarrow 27^{\circ}\text{C} \rightarrow 29^{\circ}\text{C} \rightarrow 28^{\circ}\text{C} \rightarrow 26^{\circ}\text{C}$

But if we randomly rearranged the data

$28^{\circ}\text{C} \rightarrow 29^{\circ}\text{C} \rightarrow 27^{\circ}\text{C} \rightarrow 26^{\circ}\text{C} \rightarrow 25^{\circ}\text{C}$

We lose the original temporal relationships.

- Why is Sequential nature important?
Current observations may depend on previous observations.
Trends can be identified only when the order is maintained.
Patterns such as seasonality and cycles depend on sequence.
Forecasting models use previous observations to predict future observations.

↳ Temporal components:

- Trend: A trend represents the long-term upward or downward movement of the data.
- Seasonality: Seasonality refers to patterns that repeat at specific and known time intervals.
- Cyclical variation: A cycle is a long term fluctuation that does not necessarily occur at a fixed intervals.
- Irregular / Random variation: These are unpredictable changes caused by unexpected events.

> Constant Frequency:

- Many Time Series datasets are collected at a constant or regular frequency. This means observations are recorded at equal intervals of time.
- for example: Daily data:
Monday → Tuesday → Wednesday → Thursday → Friday
Monthly data:
January → February → March → April → May → June
Yearly data:
2023 → 2024 → 2025 → 2026 → 2027 → 2028 → 2029 → 2030
- Constant frequency makes it easier to compare observations, identify trends, detect seasonal patterns, Apply statistical and forecasting models.

> Dynamic Nature:

- Time Series data is dynamic, meaning its value can be change continuously or periodically over time.
- Unlike static data, which may represent information at a single point in time, time series data shows how something changes over the time.
- for example: Consider the population of a city
2022 → 10 lakh, 2023 → 10.5 lakh, 2024 → 11 lakh, 2025 → 11.6 lakh
- This dynamic nature allows us to study about:
Growth: is the value increasing
Decline: is the value decreasing
Fluctuation: Does the value move up or down
Seasonality: Does the value repeat

2. Time Series Analysis

- Time Series Analysis is a statistical method used to study data is collected over a period of time. It helps us to understand how a variable has changed in the past and use that information to predict what may happen in the future.

- For example, a company may have monthly sales data:

Month	Sales
January	₹ 50,000
February	₹ 55,000
March	₹ 60,000
April	₹ 58,000
May	₹ 65,000

- Understanding the Past:

The first major purpose of Time Series Analysis is to understand what happened in the past.

Historical time series data can contain different patterns, such as:

a. Identifying Trends

A trend shows the long-term direction of the data.

For example:

₹ 50,000 → ₹ 55,000 → ₹ 60,000 → ₹ 65,000

The sales are generally increasing, therefore, the data has an upward direction.

b. Identifying Seasonal Patterns

Time series analysis can identify repeating patterns at a particular time.

For example: An ice-cream shop may have:

- High sales during summer.
 - Lower sales during winter.
- If this pattern occurs every year, it is called Seasonality.

c. Identifying Cycles

Time series analysis can also help identify long-term fluctuations.

For example, an economy may experience:

Economic growth → Slowdown → Recession → Recovery
→ Growth

These movements can help analysts understand the broader behaviour of the data.

d. Identifying Unusual Events

Time series analysis help identify observations that are significantly different from normal behaviour.

For example, Suppose a company normal daily sales are around 1 lakh but suddenly sales fall to 20000.

Analysis can help determine whether this unusual change was caused by a technical problem, a strike, a natural disaster or anything else.

e. Understanding Relationships Over Time

Time series analysis can help us to understand how current observations relate to previous observations.

For example: Yesterday Sales → Today Sales → Tomorrow Sales.

● Forecasting the future

The second major purpose of time series analysis is forecasting. forecasting means using historical and current time series information to estimate what is likely to happen in future.

for example, suppose a company has the following monthly sales:

January: ₹ 50,000

February: ₹ 55,000

March: ₹ 60,000

April: ₹ 65,000

The company may use the historical pattern to estimate:

May: ₹ 70,000 (Forecast)

1. Why is Forecasting important?

a. Sales Forecasting: A business can predict future sales. for example: "Based on previous sales pattern, we expect sales to increase next month".

b. Demand Forecasting: A retailer can estimate how many products customers may purchase.

for example: If demand of umbrellas is more in rainy days, the retailer can stock more umbrellas beforehand.

c. Financial Forecasting: Companies can forecast Revenue, expenses, profit and cash flow.

● How The concepts are connected?

Historical data → Analysis → Identify patterns → Forecast futures.

3. Time Series of Decomposition.

- Trends
- Seasonality
- Cyclical variations
- Irregular / Random variations

4. Code of Time Series Analysis

• Time Series Data

```
import yfinance as yf
import matplotlib.pyplot as plt
import warnings
```

```
warnings.filterwarnings('ignore')
```

```
stock_data = yf.download('AAPL', start='2024-01-01')
```

```
stock_data.head(10)
```

```
plt.figure(figsize=(12,6))
```

```
plt.plot(stock_data.index, stock_data['close'], label='Close  
Price', color='red')
```

```
plt.title('AAPL Stock Price Over Time')
```

```
plt.xlabel('Date')
```

```
plt.ylabel('Price (USD)')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

• Classical Time Series Decomposition

```
from statsmodels.tsa.seasonal import seasonal_decompose  
decomposition_additive = seasonal_decompose(stock_data  
[ 'Close' ], model = 'additive', period = 30)
```

```
trend_additive = decomposition_additive.trend
```

```
seasonal_additive = decomposition_additive.seasonal
```

```
residual_additive = decomposition_additive.resid
```

```
plt.figure(figsize = (14, 10))
```

```
plt.subplot(411)
```

```
plt.plot(stock_data[ 'Close' ], label = 'Original', color = 'red')
```

```
plt.legend(loc = 'upper left')
```

```
plt.subplot(412)
```

```
plt.plot(trend_additive, label = 'Trend', color = 'blue')
```

```
plt.legend(loc = 'upper left')
```

```
plt.subplot(413)
```

```
plt.plot(seasonal_additive, label = 'Seasonal', color = 'black')
```

```
plt.legend(loc = 'upper left')
```

```
plt.subplot(414)
```

```
plt.plot(residual_additive, label = 'Residual', color = 'orange')
```

```
plt.legend(loc = 'upper left')
```

```
plt.tight_layout()
```

```
plt.show()
```

● STL Decomposition using LOESS

- > STL stands for Seasonal-Trend decomposition using LOESS. It's method for breaking a time series into three main components:

$$Y_t = T_t + S_t + R_t$$

Where Y_t = Observed time series value

T_t = Trend Component

S_t = Seasonal Component

R_t = Remainder/Residual Component

- > LOESS means locally Estimated Scatterplot Smoothing.

- > STL is a more flexible decomposition method.

- > Advantages of STL are:

- Handles Changing trends.
- Can handle changing seasonal patterns
- more robust to outliers
- Highly flexible

● Code of STL Decomposition

from statsmodels.tsa.seasonal import STL

stl = STL(stock_data['Close'], period=30)

result = stl.fit()

plt.figure(figsize=(14,10))

plt.subplot(411)

plt.plot(result.observed, labels='Original', color='red')

plt.legend(loc='upper left')

```

plt.subplot(412)
plt.plot(result.trend, label = 'Trend', color = 'blue')
plt.legend(loc = 'upper left')
plt.subplot(413)
plt.plot(result.seasonal, label = 'Seasonal', color = 'green')
plt.legend(loc = 'upper left')
plt.subplot(414)
plt.plot(result.resid, label = 'Residual', color = 'black')
plt.legend(loc = 'upper left')
plt.tight_layout()
plt.show()

```

● STL vs Classical Decomposition

Classical

- > Less flexible
- > Moving averages
- > Basic decomposition
- > Less robust
- > fixed seasonal structures
- > Can handle both decomposition such as additive and multiplicative.

STL

- > Highly flexible
- > LOESS
- > Advanced decomposition
- > More robust
- > Can handle changing patterns
- > Handle only additive decomposition.

● Stationarity in Time Series

- > A Stationarity Time series has statistical properties that remains approximately constant over time.
- > The behaviour of the series does not fundamentally change as time passes.

> A stationary series generally has:
constant variance, constant mean, constant or stable autocorrelation structure.

> Example:

Non-stationary Examples

100 120 140 160 180 200

Mean keeps increasing so it's not constant.

Stationary Examples

102 98 101 99 103 97

Values are fluctuate around a relatively stable mean.

● Why do we need Stationarity in Time Series?

- > makes prediction easier because it assumes same statistical properties throughout the time.
- > Forecasting models require a stationary data like ARIMA, SARIMA etc because if the data is strongly non-stationary then models relationships can be unreliable.

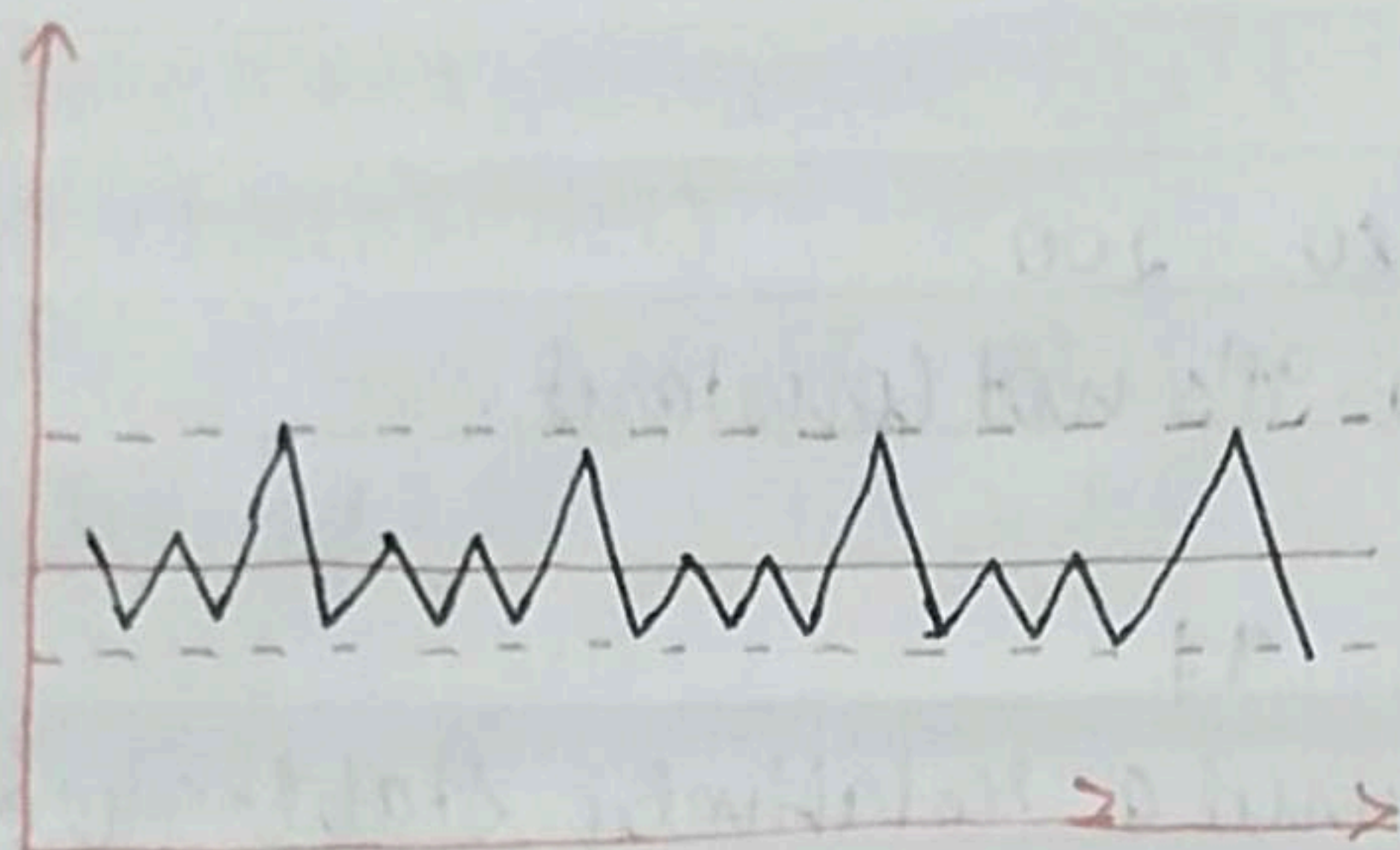
● Types of Stationarity

1. Weak Stationarity:

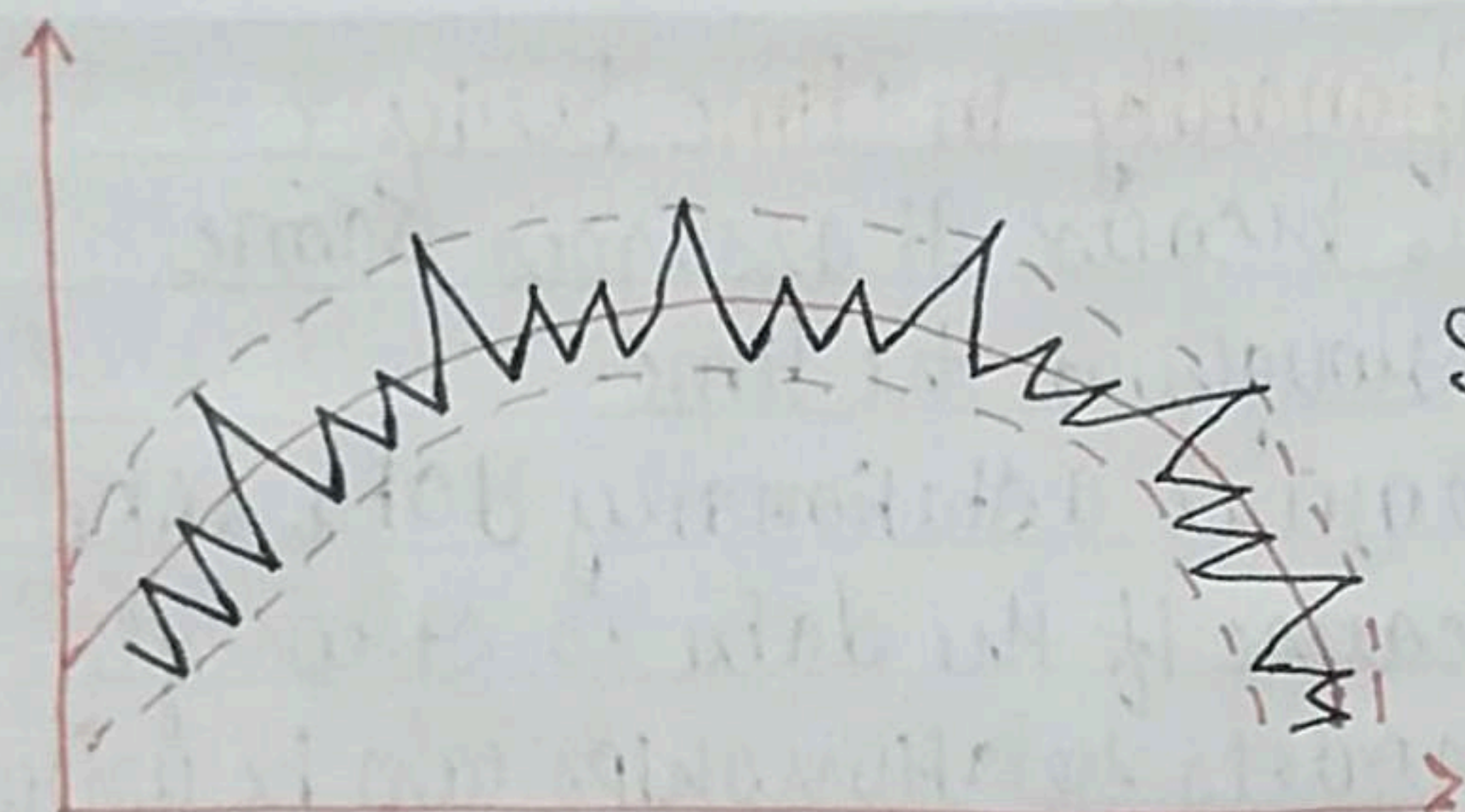
- > A series is weak stationarity only if it follows constant mean, variance and stable autocorrelation structure.
- > Joint distribution can be changed throughout the period.

2. Strict Stationarity:

- > exhibits the properties of weak stationarity.
- > Joint distribution remains unchanged when shifted along any period of time.



Stationary Variance
Stationary Mean
Stationary Variance



Stationary Variance
Non Stationary Mean

- > for different forecasting models & short period of data then use weak stationarity.
- > modeling entire distribution of data then use strict stationarity.

Testing for Stationarity

1. Augmented Dickey-Fuller Test (ADF) (H_0 is non-stationary)

> The ADF test tests for a unit root.

Simplified interpretation:

Null Hypothesis H_0 : The series is non-stationary.

Alternate Hypothesis H_1 : The series is stationary.

if $P\text{-value} < 0.05$ we generally reject H_0 therefore evidence suggests that series is stationary.

else $P\text{-value} > 0.05$ we don't reject H_0 therefore the series may be non-stationary.

2. ~~ADF~~ KPSS Test (H_0 is stationary)

> KPSS has the opposite null-hypotheses interpretation from ADF Test.

KPSS

H_0 : series is stationary

H_1 : series is non-stationary

therefore:

if $P\text{-value} > 0.05$ fails to reject H_0 therefore the series may be stationary.

While $P\text{-value} < 0.05$ we reject H_0 therefore evidence suggests that series is non-stationary.

● Code (ADF Test)

```
from statsmodels.tsa.stattools import adfuller
adf_test = adfuller(stock_data['Close'])
print('ADF Test Result:')
print(f'ADF Statistics: {round(adf_test[0], 3)}')
print(f'p-value: {round(adf_test[1], 3)}')
print('Critical Values')
for key, value in adf_test[4].items():
    print(f'{key}: {round(value, 3)}')
```

● Code (KPSS Test)

```
from statsmodels.tsa.stattools import kpss
kpss_test = kpss(stock_data['Close'], regression='t')
print('KPSS Test Results:')
print(f'KPSS Statistics: {round(kpss_test[0], 2)}')
print(f'p-value: {kpss_test[1]}')
print('Critical Values')
for key, value in kpss_test[3].items():
    print(f'{key}: {value}')
```

3. Kolmogorov-Smirnov Test

- > The KS Test compares probability distributions. It can be used to determine whether a sample is consistent with a specified distribution or whether two samples come from same distribution.
- > Compare the cumulative distribution function of two samples.
- > if $p > 0.05$ then there is no difference in distribution that's mean data is completely strict

Code (Kolmogorov-Smirnov Test)

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import ks_2samp
np.random.seed(0)
n = 500
strict_stationary_series = np.random.normal(0, 1, n)
non_strict_stationary_series = np.concatenate([
    np.random.normal(0, 1, n//2),
    np.random.normal(0, 2, n//2)
])
```

1)

```
def ks_test_stationary(series):
    split = len(series) // 2
    series_first_half = series[:split]
    series_second_half = series[split:]
    stat, p_value = ks_2samp(series_first_half,
                             series_second_half)
    return stat, p_value
ks_stat_strict, ks_pvalue_strict = ks_test_stationary(
    strict_stationary_series)
ks_stat_non_strict, ks_pvalue_non_strict = ks_test_stationary(
    non_strict_stationary_series)
plt.figure(figsize=(14, 6))
print("Stationary Series")
print("KS statistic:", ks_stat_strict)
print("P-value:", ks_pvalue_strict)
```

```
print("In Non-Stationary Series")  
print("KS Statistic:", ks_stat_non_strict)  
print("P-value:", ks_pvalue_non_strict)
```

```
plt.figure(figsize=(14, 6))
```

```
plt.plot(  
    nonstationary_series,  
    label="Stationary Series"  
)
```

```
plt.plot(  
    non_stationary_series,  
    label="Non-Stationary Series"  
)
```

```
plt.xlabel("Time")
```

```
plt.ylabel("Value")
```

```
plt.title("Stationary vs Non-Stationary Time Series")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

• Making Time Series Data to Non-Stationary data

1 Differencing:

Differencing calculates the difference between consecutive observations.

$$y_t' = y_t - y_{t-1} \quad \text{whereas } y_t' \rightarrow \text{first order differentiation}$$

$y_t \rightarrow$ current observation

$y_{t-1} \rightarrow$ Past observation.

Example

Original: First difference:

100

110

125

140

10

15

15

Python:

```
df["diff"] = df["Sales"].diff()
```

First-Order differencing
`series.diff(1)`

Second-Order differencing
`series.diff(2)`

Differencing is extremely important for ARIMA.

2. Transformation

Transformation changes the scale or mathematical representation of the data.

useful only when:

- > Variance changes with level.
- > Data is heavily skewed.
- > Growth is exponential
- > Large value dominates small values

There are Three types of Transformation which are as follows

a) Log Transformation

$$y' = \log(y)$$

useful for positively skewed data and multiplicative relationships.

Python:

```
df["log-Sales"] = np.log(df["Sales"])
```

b) Power Transformation

General form:

$$y^\lambda$$

Where λ controls the transformation.

c) Box-Cox

Box-Cox automatically searches for a suitable power transformation

It is combination of both log and Power transformation.

Python:

```
from scipy.stats import boxcox  
transformed, lam = boxcox(series)
```

• Deseasoning & Seasonal Adjustment

Deseasoning: Remove the trend from a time series.

Example: Observed $\rightarrow$ Remove trend $\rightarrow$ Residual Fluctuation

Seasonal: Remove recurring seasonal patterns.

For Example: Monthly Sales $\rightarrow$ Remove yearly seasonal patterns $\rightarrow$ Seasonally adjusted Sales.

Useful only when you want to analyze the underlying economic / business movement without seasonal effects

• Code of Making Data Stationary

```
def adf_kol(series):
```

```
    result = adfuller(series)
```

```
    print('ADF Statistics:', result[0])
```

```
    print('p-value:', result[1])
```

```
    for key, value in result[4].items():
```

```
        print('Critical Value (%s): %.3f' % (key, value))
```

```
def kpss_test(series):
```

```
prices_diff result = kpss(series, regression='c')
```

```
    print('KPSS Statistics:', result[0])
```

```
    print('p-value:', result[1])
```

```
    for key, value in result[3].items():
```

```
        print('Critical Value (%s): %.3f' % (key, value))
```

```
prices = stock_data['Close']
```

```
adf_kol(prices)
```

```
prices
```

```
prices.diff()
```

```
import numpy as np
```

```
from scipy import stats
```

```
prices_log = np.log(prices)
```

```
prices_sqrt = np.sqrt(prices)
```

```
prices_boncom & lam = stats.boncom(prices[prices > 0])
```

```
from scipy import signal
```

```
trend = np.polyfit(np.arange(len(prices)), prices, 1)
```

```
trendline = np.polyval(trend, np.arange(len(prices)))
```

```
prices_dettrended = prices - trendline
```

```
window = 12
```

```
prices_ma = prices.rolling(window = window).mean()
```

```
prices_dettrended = prices - prices_ma
```

```
prices_dettrended = prices_dettrended.dropna()
```

```
from statsmodels.tsa.seasonal import seasonal_decompose  
decomposition = seasonal_decompose(prices, model =  
    'additive', period = 30)
```

```
prices_adjusted = prices / decomposition.seasonal
```

```
prices_adjusted = prices_adjusted.dropna()
```

● White Noise & Random Walk

A white-noise series is essentially unpredictable random variation

characteristics:

- > Mean approximately zero
- > constant variance
- > No meaningful autocorrelation

Example:

-0.3 0.8 -0.1 0.2 -0.7

A good forecasting model should ideally leave white-noise residuals.

A random walk can be represented as:

$$y_t = y_{t-1} + \epsilon_t$$

Today value equals yesterday's value plus random shock

Example:

100 102 101 105 104 110

Random walks are generally non-stationary.

● Code

```
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.stats.diagnostic import acorr_ljungbox
```

```
np.random.seed(0)
```

```
n = 1000
```

white_noise = np.random.normal(0,1,n)

random_shocks = np.random.normal(0,1,n)

random_walk = np.cumsum(random_shocks)

plt.figure(figsize=(14,6))

plt.subplot(2,1,1)

plt.plot(white_noise, color='red')

plt.title('White Noise')

plt.subplot(2,1,2)

plt.plot(random_walk, color='blue')

plt.title('Random Walk')

plt.tight_layout()

plt.show()

print("Ljung-Box Test for White Noise:")

lb_test_white_noise = acorr_ljungbox(white_noise, lags=[10],
return_df=True)

print(lb_test_white_noise)

print("Ljung-Box Test for Random Walk:")

lb_test_random_walk = acorr_ljungbox(random_walk,
lags=[10], return_df=True)

print(lb_test_random_walk)

● Time Series Forecasting Models

- 1) Autoregressive (AR)
- 2) Moving Average (MA)
- 3) Autoregressive Moving Average (ARMA)
- 4) Autoregressive Integrated Moving Average (ARIMA)
- 5) Seasonal Autoregressive Integrated Moving Average (SARIMA)
- 6) Vector Autoregressive (VAR)
- 7) Vector Moving Average (VMA)
- 8) Vector Autoregressive Moving Average (VARMA)
- 9) Vector Autoregressive Integrated Moving Average (VARIMA)

1) Autoregressive (AR) model

It predicts the current value using previous values.

For AR(1): $y_t = c + \phi_1 y_{t-1} + \epsilon_t$

AR(2): $y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \epsilon_t$

The number inside AR(P) is the number of previous lags used.

Example:

Today's Sales $\leftrightarrow$ Yesterday's Sales $\leftarrow$ 2 days ago Sales

Suppose we have:

Month : Jan Feb Mar Apr May

Sales : 100 105 110 108 115

$$AR(1) : y_t = c + \phi_1 y_{t-1} + \epsilon_t$$

Suppose: $c = 10$ and $\phi_1 = 0.9$ Now predict month June

Previous value = May = 115

$$\begin{aligned} y_{\text{June}} &= 10 + (0.9)(115) \\ &= 10 + 103.5 \\ &= 113.5 \end{aligned}$$

2.3 Moving Average (MA) Models

Don't confuse this with moving average smoothing technique.

In an MA models, the current value depends on past forecast errors/shocks.

$$MA(1) : y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1}$$

$$MA(2) : y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2}$$

Example:	Jan	Feb	Mar	Apr	May
Month:	100	105	110	108	115
Actual:	98	107	108	110	113
Predicted:	+2	+2	+2	+2	+2
Error:					

for an MA(1) model:

$$y_t = \mu + \theta_1 \epsilon_{t-1} + \epsilon_t$$

Suppose: $\mu = 110$, $\theta_1 = 0.5$

Previous error: $\epsilon_{t-1} = 2$

Then:

$$\begin{aligned} y_{\text{June}} &= 110 + (0.5)(2) \\ &= 111 \end{aligned}$$

3) Autoregressive Moving Average (ARMA)

ARMA combines both AR and MA models.

That means it's used to predict current value based on past value and past forecast errors. It uses: previous values and previous errors

Suppose: ARMA(1,1) use MA Models Example

$$\text{Formula: } Y_t = C + \phi_1 Y_{t-1} + \theta_1 \epsilon_{t-1} + \epsilon_t$$

Suppose:

$C = 10$, $\phi = 0.8$, $\theta = 0.3$, previous sales = 115, previous error = 2

$$Y_t = 10 + (0.8)(115) + (0.3)(2)$$

$$= 10 + 92 + 0.6$$

$$Y_{\text{june}} = 102.06$$

4) Autoregressive Integrated Moving Average (ARIMA)

It has ARIMA(p,d,q).

Where:

Parameter

Meaning

p

AR Order

d

Differencing Order

q

MA Order

Example: ARIMA(1,1,1)

Suppose original data is:

Month	Jan	Feb	Mar	Apr	May
Sales	100	110	125	140	160

This is clearly increasing $\rightarrow$ likely non-stationary.

first difference

$$y'_t = y_t - y_{t-1}$$

Month	Jan	Feb	Mar	April	May
Sales	100	110	125	140	160
Difference	-	10	15	15	20

Now we model the differenced series is

10, 15, 15, 20

Suppose our ARIMA model on the differenced data is

$$y'_t = 2 + 0.5y'_{t-1} + 0.2e_{t-1}$$

Suppose:

Previous difference is = 20

Previous error is = 2

$$y'_t = 2 + 0.5(20) + 0.2(2)$$

$$= 2 + 10 + 0.4$$

$$= 12.4$$

Now add this to previous original value

$$y_{\text{June}} = 160 + 12.4$$

$$= 172.4$$

5) Seasonal Autoregressive Integrated Moving Average (SARIMA)

SARIMA basically ARIMA + Seasonality.

Notation:

$$\text{SARIMA}(p, d, q)(P, D, Q, s)$$

Let's use a monthly sales Example

Month	Sales
Jan 2024	100
Feb 2024	110
Mar 2024	120
...	...
Dec 2024	200
Jan 2025	120
Feb 2025	130
Mar 2025	140
...	...

Since data is monthly: $s = 12$

so we might use:

SARIMA (1,1,1) (1,1,1,12)

Meaning:

Non-Seasonal

Seasonal

$p = 1 \rightarrow$ AR

$p = 1 \rightarrow$ Seasonal AR

$d = 1 \rightarrow$ Ordinary differencing

$D = 1 \rightarrow$ Seasonal differ

$q = 1 \rightarrow$ MA

$Q = 1 \rightarrow$ Seasonal MA

$s = 12 \rightarrow$ Yearly

Seasonality

We can calculate:

$$y_t - y_{t-12}$$

for example:

Jan 2025 = 120, Jan 2024 = 100

Seasonal difference: $120 - 100 = 20$

This captures the difference from the same month last year.

6. Vector Autoregression (VAR)

Now we move to multivariate time series
suppose we have:

Months	Sales	Advertising
Jan	100	20
Feb	110	22
Mar	125	25
Apr	120	24
May	140	30

unlike AR, we are not forecasting only one variable using
its own history.

VAR allows multiple variables to influence one another.

for example:

$$\text{Sales}_t = C_1 + \phi_{11} \text{Sales}_{t-1} + \phi_{12} \text{Advertising}_{t-1}$$

$$\text{Advertising}_t = C_2 + \phi_{21} \text{Sales}_{t-1} + \phi_{22} \text{Advertising}_{t-1}$$

Suppose:

$$\text{Sales}_t = 10 + 0.5 \text{Sales}_{t-1} + 2 \text{Advertising}_{t-1}$$

for June: Previous Sales = 140, Previous Advertising = 30

$$\text{Sales} = 10 + (0.5)(140) + (2)(30)$$

$$= 10 + 70 + 60 = 140$$

$$\text{Predicted Sales} = 140$$

7. Vector Moving Average (VMA)

VMA is the multivariate equivalent of MA.

Instead of using only previous errors from one variable,
we use errors from multiple variables.

Suppose, we have:

$$\text{Sales error} = 4, \text{ Advertising error} = -2$$

$$\text{Sales}_t = 100 + 0.5e_{\text{Sales},t-1} + 0.2e_{\text{Ad},t-1}$$

$$\text{Sales}_t = 100 + (0.5)(4) + (0.2)(-2)$$

$$= 100 + 2 - 0.4$$

$$= 101.6$$

8. Vector Autoregressive Moving Average (VARMA)

VARMA combines VAR + VMA

Suppose we have: Sales, Advertising model:

VARMA(1,1)

It considers: previous values and previous errors.
for example:

$$\text{Sales}_t = 10 + 0.5 \text{Sales}_{t-1} + 0.2 \text{Advertising}_{t-1} + 0.3 e_{\text{Sales},t-1} + 0.1 e_{\text{Ad},t-1}$$

$$\text{Sales}_t = 10 + (0.5)(140) + (0.2)(30) + (0.3)(4) + (0.1)(-2)$$

$$= 10 + 70 + 6 + 1.2 - 0.2$$

$$= 87$$

Predicted Sales = 87

9. Vector Autoregressive Integrated Moving Average (VARIMA)

VARIMA extends VARMA by adding integration / differencing.

Think: VAR → VARMA → VARIMA

for example: VARIMA(p,d,q)

Suppose Sales and Advertising are both non-stationary

Month	Sales	Advertising
Jan	100	20
Feb	110	22
Mar	125	25
Apr	140	29
May	160	32

Difference both:

$$\begin{aligned}\text{Sales} &: 110 - 100 = 10 \\ &125 - 110 = 15 \\ &140 - 125 = 15 \\ &160 - 140 = 20\end{aligned}$$

$$\begin{aligned}\text{Advertising} &: 22 - 20 = 2 \\ &25 - 22 = 3 \\ &29 - 25 = 4 \\ &32 - 29 = 3\end{aligned}$$

Now the model works on

Sales difference: 10, 15, 15, 20

Advertising differences: 2, 3, 4, 3

Then it uses relationships between these multiple differenced series.

● Granger Causality Test

Granger causality examines whether past values of one time series provide useful information for predicting another.

Example: Advertising $\rightarrow$ Future Sales

if past advertising values improve the prediction of sales, advertising may Granger-cause sales.

Important:

Granger ~~causality~~ causality does not necessarily mean true causal relationship.

It means:

Past value of X contain predictive information about Y.

● Time Series Forecasting Models Using Python

● AR Model

```
stock_data = yf.download('AAPL', start = '2023-01-01')
```

```
apple_price_stationary = stock_data['close'].diff().dropna()
```

```
train_data, test_data = apple_price_stationary[:-30], apple_price_stationary[-30:]
```

```

from statsmodels.tsa.ar_model import AutoReg
from sklearn.metrics import mean_squared_error
import numpy as np
import warnings
warnings.filterwarnings('ignore')

model = AutoReg(train_data, lags=30)
model_fit = model.fit()

predictions = model_fit.predict(
    start = len(train_data),
    end = len(train_data) + len(test_data) - 1,
    dynamic = False)

plt.figure(figsize=(10,5))
plt.plot(test_data.index, test_data, label='Test data')
plt.plot(test_data.index, predictions, color='red', linestyle
        = '--', label='Predicted Prices')

plt.title('Apple Stock Prices: Actual vs Predicted')
plt.xlabel('Date')
plt.ylabel('Close Price')
plt.legend()
plt.show()

rmse = round(np.sqrt(mean_squared_error(test_data,
        predictions)), 2)

print('RMSE:', rmse)

```

• MA Model

```
from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(train_data, order = (0, 0, 30))
model_fit = model.fit()
predictions = model_fit.predict(
    start = len(train_data),
    end = len(train_data) + len(test_data) - 1,
    dynamic = False)

plt.figure(figsize = (10, 5))
plt.plot(test_data.index, test_data, label = 'Test data')
plt.plot(test_data.index, predictions, color = 'red', linestyle = '--',
    label = 'Predicted Prices')

plt.title('Apple Stock Prices: Actual vs Predicted')
plt.xlabel('Date')
plt.ylabel('Close Price')
plt.legend()
plt.show()
rmse = round(np.sqrt(mean_squared_error(test_data,
    predictions)), 2)

print('RMSE:', rmse)
```

• ARMA Model

```
model = ARIMA(train_data, order = (7, 0, 7))
model_fit = model.fit()
predictions = model_fit.predict(
    start = len(train_data),
    end = len(train_data) + len(test_data) - 1,
    dynamic = False)
```

```

plt.figure(figsize=(10,5))
plt.plot(test_data.index, test_data, label='Test data')
plt.plot(test_data.index, predictions, color='red', linestyle
         = '--', label='Predicted Prices')
plt.title('Apple Stock Prices: Actual vs Predicted')
plt.xlabel('Date')
plt.ylabel('Close Price')
plt.legend()
plt.show()
rmse = round(np.sqrt(mean_squared_error(test_data,
                                         predictions)), 2)
print('RMSE:', rmse)

```

● ARIMA Model

```

model = ARIMA(train_data, order=(7,1,7))
model_fit = model.fit()
predictions = model_fit.predict(start=len(train_data),
                                end=len(train_data)+len(test_data)-1,
                                dynamic=False)

plt.figure(figsize=(10,5))
plt.plot(test_data.index, test_data, label='Test Data')
plt.plot(test_data.index, predictions, color='red', linestyle
         = '--', label='Predicted Prices')
plt.title('Apple Stock Prices: Actual vs Predicted')
plt.xlabel('Date')
plt.ylabel('Close Price')
plt.legend()
plt.show()

```

• SARIMA Model

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
P, d, q = 7, 1, 7
```

```
P, D, Q, s = 1, 1, 1, 45
```

```
model = SARIMAX(train_data, order=(P, d, q), seasonal_order=(P, D, Q, s))
```

```
model_fit = model.fit()
```

```
predictions = model_fit.predict(start=len(train_data),  
                                end=len(train_data)+len(test_data)-1,  
                                dynamic=False)
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(test_data.index, test_data, label='Test data')
```

```
plt.plot(test_data.index, predictions, color='red', linestyle='--',  
         label='Predicted Prices')
```

```
plt.title('Apple Stock Prices: Actual vs Predicted')
```

```
plt.xlabel('Date')
```

```
plt.ylabel('Close Price')
```

```
plt.legend()
```

```
plt.show()
```

```
rmse = round(np.sqrt(mean_squared_error(test_data,  
                                         predictions)), 2)
```

```
print('RMSE:', rmse)
```

• Vector Models

```
tsla_data = yf.download('TSLA', start='2023-01-01')
```

```
stock_data['TSLA_Close'] = tsla_data['Close']
```

```
stock_data['AAPL_Close'] = stock_data['Close'].shift()
```

```
stock_data.dropna(inplace=True)
```

```
from statsmodels.tsa.stattools import grangercausalitytests  
grangercausalitytests(stock_data[['AAPL_Close', 'TSLA_Close']])
```

```
print()
```

```
dropna(), maxlag=[14])
```

```
tsla_data
```

```
stock_data
```

```
data = stock_data[['APPL-Close', 'TSLA-Close']].diff().dropna()
```

```
train_data, test_data = data[: -14], data[-14:]
```

```
# VAR Model
```

```
from statsmodels.tsa.api import VAR, VARMAX
```

```
model = VAR(train_data)
```

```
result = model.fit(maxlags=7)
```

```
predictions = result.forecast(train_data.values[-result.k_ar:], steps=len(test_data))
```

```
predictions = pd.DataFrame(predictions, index=test_data.index, columns=test_data.columns)
```

```
plt.figure(figsize=(10,6))
```

```
plt.plot(test_data.index, test_data['TSLA-Close'], label='Actual Price', color='blue')
```

```
plt.plot(test_data.index, predictions['TSLA-Close'], label='VAR Forecast', color='red')
```

```
plt.legend()
```

```
plt.title('Tesla Stock Price Prediction')
```

```
plt.show()
```

```
rmse = round(np.sqrt(mean_squared_error(test_data['TSLA-Close'], predictions['TSLA-Close'])), 2)
```

```
print('RMSE:', rmse)
```

VMA Model

```
model = VARMAX(train_data, order = (0, 14))
```

```
result = model.fit()
```

```
predictions = result.predict(  
    start = len(train_data), end = len(train_data) +  
    len(test_data) - 1, dynamic = False)
```

```
plt.figure(figsize = (10, 6))
```

```
plt.plot(test_data.index, test_data['TSLA_close'], label =  
    'Actual Price', color = 'blue')
```

```
plt.plot(test_data.index, predictions['TSLA_close'], label =  
    'VMA Forecast', color = 'red')
```

```
plt.legend()
```

```
plt.title('Tesla Stock Price Prediction')
```

```
plt.show()
```

```
rmse = round(np.sqrt(mean_squared_error(test_data  
    ['TSLA_close'], predictions['TSLA_close'])), 2)
```

```
print('RMSE:', rmse)
```

VARIMA Model

```
model = VARMAX(train_data, order = (14, 14))
```

```
result = model.fit()
```

```
prediction = result.predict(start = len(train_data), end =  
    len(train_data) + len(test_data) - 1, dynamic = False)
```

```
plt.figure(figsize = (10, 6))
```

```
plt.plot(test_data.index, test_data['TSLA_close'], label = 'Actual  
Price', color = 'blue')
```

```
plt.plot(test_data.index, predictions['TSLA_close'], label =  
    'VARIMA Forecast', color = 'red')
```

```
plt.legend()
```

```
plt.title('Tesla Stock Price Prediction')
```

```
plt.show()
```